1. *Define the hook function.* We create a function with a descriptive name that reflects the hook's purpose.
2. *Identify state management needs.* We capture what state the hook needs to manage internally and what it needs to expose externally. It makes sense to keep most of the state internal and only return values that other components need to use.
3. *Create helper functions, if needed.* Sometimes hooks can become complex. We can improve this by defining functions that can encapsulate specific tasks related to state updates or data manipulation. We also consider accepting helper functions as arguments, allowing us to use different implementations on demand.
4. *Implement core functionality.* The core functionality of your hook depends on its purpose. It might involve handling user interactions (like form submissions), managing data fetching (e.g., from APIs), or manipulating UI state (like showing/hiding modals).
5. *Return the hook values.* A key consideration is what values your hook needs to expose. A hook should return an object containing the state variables it manages and any functions it provides for interacting with that state. This allows any component to call the hook to access and manipulate the relevant data and functionalities.

Understanding these core steps is essential before we review the specific implementation details. Let's break down how we used the general guidelines for building custom hooks to create the specific `useChatFormSubmit` hook:

1. *Define the hook function.* Custom hooks begin with use to signal their role and purpose. `useChatFormSubmit` is named to clearly indicate it handles chat form submissions.
2. *Identify state management needs.* Hooks manage dynamic data within components. `useChatFormSubmit` tracks chat messages, the current user input, and the submission's loading status.
3. *Create helper functions (and dependencies).* Hooks often rely on helper functions for complex or external tasks. `useChatFormSubmit` uses a helper, `getAssistantResponse` (passed as an argument), to fetch the LLM response from the backend.
4. *Implement core functionality.* This is the hook's main logic. `useChatFormSubmit` manages the form submission, updates all past messages for the `<ChatList/>` display, and exposes a loading state for user feedback.
5. *Return the hook values.* Hooks provide values (state and functions) for components to use. `useChatFormSubmit` returns an object with
   - *Messages*—Conversation history
   - `isLoading`—Current loading status
   - `handleSubmit`—Function to submit the form
   - `inputValue`—Current input text
   - `setInputValue`—Function to update input text